

## **Desenvolvimento e Análise Prática do Algoritmo de Dijkstra em C**

Aluno: Ana Beatriz Abrantes Da Silva

Professor: Gioliano Bertoni

Curso: Engenharia de Software

Disciplina: Estrutura de Dados

Engenharia de Software

Saquarema – 2026

## 1. INTRODUÇÃO

A Teoria dos Grafos constitui um dos pilares fundamentais da Ciência da Computação, servindo como modelo matemático para a representação de sistemas complexos e relações entre objetos discretos. Seja na otimização de rotas em sistemas de transporte (como GPS), no roteamento de dados na internet ou na análise de conexões em redes sociais, os grafos oferecem uma estrutura robusta e versátil para o armazenamento e processamento de informações.

O estudo dessas estruturas permite compreender a eficiência de algoritmos essenciais para encontrar caminhos ótimos. A escolha da representação computacional e do algoritmo adequado impacta diretamente o consumo de recursos e a performance de execução da aplicação. Neste contexto, o presente trabalho dedica-se à exploração teórica e prática dos grafos ponderados. Apresenta-se a implementação do Algoritmo de Dijkstra na linguagem C através de um arquivo fonte único (main.c), com o intuito de demonstrar o mecanismo de busca pelo caminho mínimo em uma estrutura de dados baseada em matriz de adjacência.

## 2. FUNDAMENTAÇÃO TEÓRICA

### 2.1 DEFINIÇÃO DE GRAFOS

Um grafo é uma estrutura composta por dois elementos os vértices e as arestas. Os vértices representam os pontos ou entidades de interesse, como cidades, computadores ou cruzamentos, e as arestas representam as linhas que conectam esses pontos, como estradas, cabos ou conexões.

Os grafos podem ser classificados de acordo com suas propriedades, como o grafo direcionado (ou dígrafo), onde as arestas possuem uma direção definida por setas, funcionando como ruas de mão única; o grafo não-direcionado, onde as conexões não possuem orientação, permitindo o tráfego livre de via dupla entre os nós, ou o grafo ponderado (com peso), onde cada aresta possui um valor numérico associado que representa o custo de travessia, como a distância em quilômetros ou o tempo de deslocamento.

## 2.2 REPRESENTAÇÃO POR MATRIZ DE ADJACÊNCIA

A forma como o grafo é armazenado na memória define diretamente a eficiência e o consumo de recursos do algoritmo. Neste projeto, utilizou-se a Matriz de Adjacência, que consiste em uma tabela bidimensional de tamanho  $V \times V$ , onde  $V$  representa o número total de vértices do grafo. Nessa estrutura, tanto as linhas quanto as colunas da matriz correspondem aos vértices do sistema. O elemento localizado na posição  $[i][j]$  da tabela guarda o valor numérico do peso da aresta que vai do vértice  $i$  para o vértice  $j$ . Quando não existe uma conexão direta entre esses dois nós, o valor armazenado nessa posição é preenchido com 0 (ou, em algumas lógicas, com um valor infinito para indicar que o caminho é inacessível). Além disso, as posições da diagonal principal da matriz, onde  $i$  é igual a  $j$ , também recebem o valor 0, representando que não há custo de deslocamento de um vértice para ele mesmo.

## 3. METODOLOGIA E ALGORITMO

As aplicações práticas desse modelo são fundamentais no cotidiano tecnológico atual. Em sistemas de GPS, como o Google Maps e o Waze, esses conceitos servem para calcular a rota mais rápida até um destino com base na distância e no trânsito. Em Redes de Computadores, o modelo é utilizado por roteadores para guiar pacotes de dados de internet pelo caminho mais rápido e estável. Do mesmo modo, ele é aplicado em redes sociais para mapear graus de separação e sugerir conexões de menor distância entre os usuários.

### 3.1 O ALGORITMO DE DIJKSTRA

Desenvolvido por Edsger Dijkstra em 1956, este algoritmo resolve o problema do caminho mínimo de origem única em grafos ponderados com pesos não-negativos. Diferente de uma Busca em Largura comum (BFS), que usa uma fila simples do tipo FIFO e funciona apenas para grafos sem peso, o algoritmo de Dijkstra adota uma estratégia de escolha imediata. Ele avalia os vizinhos e foca sempre em processar primeiro o vértice disponível que possui a menor distância acumulada até a origem, controlando o fluxo por meio de um conceito análogo a uma fila de prioridades para garantir a melhor rota possível ao final da execução.

### 3.2 IMPLEMENTAÇÃO EM C

A implementação foi realizada de forma direta em um único arquivo chamado main.c. O grafo de teste possui 5 vértices e foi mapeado de forma estática no código. A implementação completa desenvolvida pode ser analisado publicamente através do repositório hospedado no GitHub através do link: <https://github.com/AnaBeatrizAbrantes/Algoritmo-Grafos>.

Abaixo Print da função Dijkstra no código:

```
20 void dijkstra(int grafo[V][V], int origem) {
21     int dist[V]; // Menores distancias
22     bool visitado[V]; // Array para mapear os vertices ja processados
23
24     for (int i = 0; i < V; i++) {
25         dist[i] = INF;
26         visitado[i] = false;
27     }
28
29     dist[origem] = 0;
30
31     for (int count = 0; count < V - 1; count++) {
32         int u = encontrarMenorDistancia(dist, visitado);
33
34         visitado[u] = true;
35
36         for (int v = 0; v < V; v++) {
37             if (!visitado[v] && grafo[u][v] && dist[u] != INF && dist[u] + grafo[u][v] < dist[v]) {
38                 dist[v] = dist[u] + grafo[u][v];
39             }
40         }
41     }
```

### 3.3 ANÁLISE DE COMPLEXIDADE

Em termos de desempenho estrutural, o algoritmo apresenta uma Complexidade de Tempo definida como  $O(V^2)$ . Esse comportamento ocorre porque a busca pelo nó de menor distância utiliza uma varredura linear simples por meio de um laço For de tamanho V, que por sua vez executa aninhado dentro do laço principal do algoritmo. Quanto à Complexidade de Espaço, o algoritmo também opera em  $O(V^2)$ , uma vez que o consumo de memória é determinado estaticamente pelo tamanho da tabela bidimensional que armazena a matriz de adjacência (grafo[V][V]) na memória principal.

## 4. RESULTADOS OBTIDOS

A compilação do arquivo main.c foi realizada no ambiente de terminal Git Bash utilizando o compilador GCC através do comando gcc main.c -o programa. A execução foi feita via ./programa.exe. No repositório público há instruções de como fazer a compilação do sistema no Readme.

A imagem abaixo mostra a saída gerada pelo sistema:

```
Ana@Jeff MINGW64 ~/Desktop/ED/Algoritmo-Grafos (main)
$ ./programa
Vertice          Distancia Minima da Origem (0)
0                0
1                3
2                2
3                5
4                6
```

O programa executou a busca partindo do vértice 0 como origem e determinou com precisão os custos mínimos para todos os nós alcançáveis do grafo, revelando o comportamento dinâmico do algoritmo. Ao analisar o vértice 1, que obteve custo 3, observa-se que, embora exista uma aresta direta conectando a origem ao nó 1 com custo 4, o algoritmo identificou que o percurso alternativo passando pelo vértice 2 acumulava um custo menor (soma do custo 2 do trecho inicial com o custo 1 do trecho seguinte).

O próprio vértice 2 foi mapeado com custo 2 por ser o vizinho mais próximo da origem, funcionando como um ponto intermediário para baratear o acesso aos demais nós. A partir dessas análises, o sistema realizou a atualização das distâncias para os vértices 3 e 4, que finalizaram com os custos ótimos de 5 e 6.

## 5. CONCLUSÃO

A elaboração deste projeto prático permitiu consolidar de maneira clara os conceitos teóricos sobre a modelagem de grafos ponderados. A escolha por uma estrutura em arquivo único facilitou a manutenção, compilação e teste imediato da lógica. O algoritmo de Dijkstra provou ser uma ferramenta de otimização extremamente robusta. A experiência evidenciou que entender o comportamento das estruturas de dados é indispensável para criar soluções de software capazes de resolver problemas reais de roteamento e logística de forma ágil e precisa.

## 6. REFERÊNCIAS

**CORMEN**, Thomas H. et al. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

**TENENBAUM**, Aaron M.; **LANGSAM**, Yedidyah; **AUGENSTEIN**, Moshe J. *Estruturas de Dados Usando C*. São Paulo: Pearson Makron Books, 1995.

**DIJKSTRA**, Edsger W. *A note on two problems in connexion with graphs*. Numerische Mathematik, v. 1, n. 1, p. 269-271, 1959.

**ZIVIANI**, Nivio. *Projeto de Algoritmos: com implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2011.